

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

Departamento de Sistemas Digitales

SolarBI Pascual: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Trabajo de consulta

Brayan Romero Dorado

Inteligencia de Negocios · Grupo 01

Docente: Ramiro Grisales Montoya

Medellín, 9 de octubre de 2026

Repositorio: github.com/romerobrayan/solarbi-romero-dorado

Parte A · Consulta

Bloque 1 · Power BI y Grafana

D1. ¿Qué es Grafana? Componentes y diferencia entre Grafana OSS y Grafana Cloud

Grafana es una plataforma de código abierto para visualizar y monitorear series de tiempo. No almacena datos: consulta el origen cada vez que se refresca. El **data source** es esa conexión; en SolarBI, PostgreSQL con TimescaleDB. Un **panel** ejecuta una consulta y la dibuja, como la potencia p_ac_kw cada 5 minutos. El **dashboard** reúne varios paneles. Las **variables**, como \$dispositivo, cambian de inversor desde una lista sin duplicar el tablero. **Alerting** evalúa reglas y avisa por un contact point (Grafana Labs, s. f.-a, s. f.-b, s. f.-f).

Grafana OSS es gratuita (AGPLv3) y la instalo en Docker. **Grafana Cloud** es el servicio administrado por Grafana Labs, con plan gratuito limitado y planes de pago (Grafana Labs, s. f.-c, s. f.-d).

D2. Tabla comparativa Power BI vs. Grafana

Criterio	Power BI	Grafana
Propósito principal	Análisis de negocio: informes interactivos y cuadros de mando para decidir.	Monitoreo y observabilidad de series de tiempo para operar.
Usuario típico	Analista BI, jefe de planta, directivo.	Operador, técnico de mantenimiento, ingeniero de datos.
Conexión a los datos	Import (copia comprimida en memoria), DirectQuery o conexión dinámica. Power Query transforma antes de cargar.	Consulta el origen en cada refresco mediante plugins de data source. No guarda copia de los datos.
Modelo semántico y lenguaje	Modelo propio con tablas, relaciones y medidas en DAX; transformaciones en lenguaje M.	Sin modelo semántico: la lógica va en la consulta del origen (SQL en PostgreSQL) y en las transformaciones del panel.
Actualización y tiempo real	En Import, actualización programada: hasta 8 al día con Pro y 48 con PPU o capacidad. DirectQuery se acerca al tiempo real. Los modelos de streaming se retiran en octubre de 2027 y Microsoft remite a Fabric Real-Time Intelligence.	Refresco automático del dashboard cada pocos segundos. Está pensado para tiempo real.
Alertas	Alertas de datos solo sobre tarjetas, KPI y medidores fijados en un dashboard del servicio. Se revisan tras cada actualización y avisan por correo, centro de notificaciones o Power Automate.	Alert rules con consultas, expresiones y umbrales, evaluadas periódicamente. Notifican por contact points (correo, Slack, Telegram, webhook) según las notification policies.

Criterio	Power BI	Grafana
Licenciamiento y costo	Desktop gratuito. Para compartir: Pro, USD 14 por usuario/mes, o Premium Per User, USD 24. Capacidad Fabric para organizaciones.	OSS gratuito (AGPLv3) y autoalojado. Cloud Free: 3 usuarios, 10.000 series activas y 14 días de retención. Cloud Pro desde USD 19/mes más consumo.
Control de acceso	Roles de área de trabajo (Admin, Member, Contributor, Viewer) y seguridad a nivel de fila (RLS) con filtros DAX.	Roles de organización (Admin, Editor, Viewer), equipos y permisos por carpeta o dashboard. No filtra filas por usuario de forma nativa.
Versionamiento	El .pbix es binario; el formato .pbip guarda modelo e informe como texto.	El dashboard es un JSON exportable que se puede cargar desde archivos (provisioning).
Despliegue en SolarBI	Power BI Desktop en Windows y publicación en el servicio en la nube.	Contenedor Docker junto a PostgreSQL, en la misma máquina.

Fuentes de la tabla: Microsoft (2024, s. f.-a, s. f.-b, s. f.-c); Grafana Labs (s. f.-a, s. f.-c, s. f.-d, s. f.-e). Precios en USD consultados el 8 de octubre de 2026.

La diferencia de fondo está en dónde vive la lógica. En Power BI los datos se modelan una vez y las cifras salen de medidas DAX; en Grafana cada panel le pregunta directamente a PostgreSQL. Para SolarBI eso significa que la potencia cada 5 minutos se sigue mejor en Grafana, mientras que la energía, el PR y el ahorro por mes se calculan mejor en Power BI.

D3. Dashboard operativo, analítico y estratégico; clasificación de los KPI

Siguiendo a Few (2006), un dashboard **operativo** muestra lo que pasa ahora para reaccionar en minutos. El **analítico** sirve para explorar tendencias y causas comparando periodos. El **estratégico** resume el desempeño frente a metas para quien decide, y cambia poco.

Para clasificar usé una regla sencilla: si el KPI exige actuar durante el turno, va en Grafana; si exige agregar, comparar o convertir a dinero, va en Power BI. Por eso la energía aparece en las dos herramientas, aunque cada una responde una pregunta distinta.

KPI	Tipo	Herramienta	Por qué
Energía generada (kWh)	Operativo (hoy) y analítico (histórico)	Grafana para la del día; Power BI para la diaria y la mensual	El operador necesita saber si hoy la planta va normal. El análisis por días y meses sale de medidas DAX sobre fact_energía_dia.
Yield (kWh/kWp) y Performance Ratio	Analítico	Power BI	Se calculan con agregados diarios: $Yf = E / P0$ y $PR = Yf / Yr$, donde Yr depende de la irradiación (Marion et al., 2005). Sirven para comparar inversores y meses.
Ahorro (COP)	Estratégico	Power BI	Convierte energía en dinero con la tarifa. Lo revisa un directivo una vez al mes frente a la meta.

KPI	Tipo	Herramienta	Por qué
Disponibilidad y alarmas	Operativo	Grafana	Una potencia en cero a mediodía exige reacción inmediata: la alert rule de 9:00 a 15:00 avisa al operador. La disponibilidad mensual puede pasar luego a Power BI.
% de datos válidos	Operativo (calidad de datos)	Grafana, con tarjeta de apoyo en Power BI	Mide la salud del pipeline después de cada carga (dq.etl_run_log). En Power BI acompaña las cifras para saber cuánto confiar en ellas.

Fuentes: Few (2006); Marion et al. (2005).

D4. Consultas de series de tiempo en Grafana con PostgreSQL

Grafana espera que la consulta devuelva una columna llamada time con la marca de tiempo, una columna numérica con el valor y, si hay varias series, una columna de texto que las separe, todo ordenado por time. \$__timeFilter(ts) se reemplaza por un filtro BETWEEN con el rango del selector de tiempo. \$__timeGroup(ts, '1h') agrupa las marcas de tiempo en intervalos; como el data source de SolarBI tiene TimescaleDB activado, Grafana lo traduce a time_bucket. \$__timeGroupAlias hace lo mismo y además nombra la columna time (Grafana Labs, s. f.-g). Para la potencia promedio por hora, la consulta del tablero promedia la potencia de Silver en grupos de una hora, solo dentro del rango elegido y para el inversor seleccionado (SELECT \$__timeGroupAlias(ts, '1h'), avg(p_ac_kw) AS potencia_promedio_kw FROM silver.lectura_5min WHERE \$__timeFilter(ts) AND dispositivo_id IN (\$dispositivo) GROUP BY 1 ORDER BY 1).

D5. Alertas: Grafana Alerting frente a las alertas de datos de Power BI

En Grafana, una **alert rule** ejecuta una consulta periódicamente y la compara con una condición. Si la condición se sostiene durante el tiempo de espera (for), pasa a *Firing* y las notification policies la envían a un **contact point**: correo, Slack o webhook. En Power BI, las alertas de datos solo existen sobre tarjetas, KPI y medidores fijados en un dashboard del servicio; se revisan tras cada actualización y avisan por correo, como máximo una vez por hora o por día (Grafana Labs, s. f.-a; Microsoft, s. f.-c). Mi regla para la planta: si la última lectura de un inversor marca 0 kW entre las 9:00 y las 15:00 durante 10 minutos, el operador recibe un correo con severidad *critical*.

Bloque 2 · Gobernanza de datos (data governance)

G1. Data governance frente a data management y data quality

Según DAMA International (2017), la **gobernanza de datos** es el ejercicio de autoridad y control sobre cómo se gestionan los datos: define quién decide, con qué políticas y cómo se mide que se cumplan. La **gestión de datos** (data management) son las prácticas que ejecutan ese plan en el día a día: modelado, almacenamiento, integración, seguridad y respaldos. La **calidad de datos** es una de esas áreas y mide si el dato sirve para su uso: completo, exacto y a tiempo. En SolarBI la gobernanza vive en el contrato de datos y sus

responsables; la gestión, en el ETL y en PostgreSQL; y la calidad, en las reglas que dejan registro en dq.rule_result después de cada carga.

G2. Roles data owner, data steward y data custodian

El **data owner** responde por el dato ante el negocio: aprueba cambios y fija el nivel de calidad aceptable. El **data steward** cuida las definiciones, unidades y reglas, y revisa la calidad después de cada carga. El **data custodian** opera la infraestructura (servidores, permisos, respaldos y ejecución del ETL) sin cambiar las definiciones (DAMA International, 2017). En SolarBI los dejé escritos en el contrato: el product owner es el data owner; el BI analyst y el data modeler actúan como stewards, y el data engineer es el custodian. Como trabajé solo asumí los tres roles, pero tenerlos separados deja claro quién aprueba cada cambio cuando el equipo crezca.

G3. El contrato de datos como instrumento de gobernanza

Un **contrato de datos** es un acuerdo versionado entre quien produce los datos y quien los consume. Debe incluir el esquema (columnas y tipos), las unidades, la frecuencia y la zona horaria, el responsable, las reglas de calidad con su nivel mínimo y la forma de anunciar cambios. En SolarBI es contracts/telemetry.yaml: el ETL lee de ahí rangos y reglas, el SLA exige 95 % de datos válidos y un cambio incompatible obliga a subir la versión mayor. Si la fuente renombra p_ac_kw sin avisar, el tablero mostraría vacíos o ceros y nadie sabría si la planta dejó de generar o si cambió el archivo. Con el contrato, el cambio se detecta en la carga (DAMA International, 2017).

G4. Seguridad a nivel de fila (RLS) en Power BI y su equivalente en Grafana

RLS limita qué filas ve cada usuario del modelo. En Power BI Desktop se crea un rol en Modelado → Administrar roles, con un filtro DAX que decide qué filas se ven; luego, en el servicio, se asignan personas al rol. Para que un responsable solo vea su sitio, uso una tabla de accesos con el correo de cada persona y su sitio, y filtro dim_sitio con los sitios del usuario conectado (USERPRINCIPALNAME() devuelve su correo). RLS solo aplica a quienes tienen rol Viewer en el área de trabajo (Microsoft, s. f.-d). Grafana no filtra filas por usuario: lo equivalente es separar por organizaciones, equipos y permisos por carpeta o dashboard (Grafana Labs, s. f.-e).

G5. Ley 1581 de 2012 y manejo responsable de la información

La Ley 1581 de 2012 protege los datos personales, es decir, la información vinculada a personas naturales determinadas o determinables. La telemetría de SolarBI (potencia, irradiancia y temperatura de un inversor) no identifica a nadie, así que no es un dato personal. Eso cambia si el tablero muestra qué operador atendió cada alarma: su nombre junto a su desempeño sí lo es, y exige informar la finalidad, pedir autorización previa, restringir el acceso y permitirle consultar, corregir o pedir que se supriman sus datos (Congreso de la República de Colombia, 2012). Mi postura: recolectar solo lo necesario, mostrar un código de operador en vez del nombre y no subir nunca credenciales ni datos personales al repositorio público.

Bloque 3 · Automatización de procesos ETL

E1. ETL frente a ELT en las capas Bronze, Silver y Gold; ¿Power Query es ETL?

En **ETL** los datos se transforman antes de llegar al destino; en **ELT** primero se cargan crudos y después se transforman dentro de la base. SolarBI sigue el patrón ELT de la arquitectura medallion (Databricks, s. f.): el CSV se carga tal cual en **Bronze**, las reglas de calidad corren en SQL dentro de PostgreSQL para pasar a **Silver**, y la agregación diaria produce **Gold**. Power Query sí cumple el papel de una herramienta ETL, porque extrae, transforma en lenguaje M y carga al modelo. El problema es que esas transformaciones quedan amarradas a un archivo de Power BI y no le sirven a Grafana. Por eso la limpieza va en la base y Power Query solo importa Gold.

E2. Carga completa, carga incremental e idempotencia con UPSERT

La **carga completa** borra y vuelve a cargar todo el destino: es simple, pero se vuelve lenta y deja la tabla vacía mientras corre. La **incremental** solo procesa lo nuevo o lo que cambió desde la última carga. Una carga es **idempotente** si correrla dos veces con la misma entrada deja la base igual que correrla una vez. En fact_energia_dia lo logro con la clave primaria (fecha_key, dispositivo_key) y un UPSERT, que inserta el día o lo actualiza si ya existe, en lugar de duplicarlo (INSERT ... ON CONFLICT DO UPDATE) (PostgreSQL Global Development Group, s. f.). Además, Bronze guarda el hash del archivo y no lo copia otra vez. El paso 3 de la Parte B lo demuestra con los conteos.

E3. cron, Apache Airflow y actualización programada de Power BI

Forma	Qué es	Qué automatiza	Cuándo conviene
cron / Programador de tareas	Planificador del sistema operativo que lanza un comando a una hora fija.	El comando completo de run_etl.py, por ejemplo cada día a medianoche.	Un solo flujo en un solo servidor, sin dependencias entre tareas. Es lo que usa SolarBI hoy.
Apache Airflow	Orquestador de código abierto que define cada flujo como un DAG escrito en Python (Apache Software Foundation, s. f.).	Varias tareas con dependencias, reintentos, historial y alertas cuando algo falla.	Cuando haya varias plantas o fuentes y pasos que dependan unos de otros.
Actualización programada de Power BI	Función del servicio que refresca un modelo importado en horarios definidos (Microsoft, s. f.-a).	Solo la copia de los datos dentro de Power BI; no ejecuta el ETL.	Para que el informe tome Gold después de cada carga. Con una base local necesita una puerta de enlace (gateway).

No compiten: cron o Airflow llenan PostgreSQL, y la actualización programada refresca Power BI cuando el ETL ya terminó.

E4. Query folding en Power Query

Query folding es la capacidad de Power Query de traducir los pasos de transformación a una sola consulta en el lenguaje del origen; con PostgreSQL, a SQL. Así el filtro y la

agrupación los hace la base de datos y a Power BI solo llegan las filas necesarias: menos memoria, menos tráfico y actualizaciones más rápidas. En Power BI Desktop se revisa con clic derecho sobre un paso → Ver consulta nativa: si muestra un SELECT, ese paso se plegó; si la opción está deshabilitada, puede que ese paso ya se ejecute en el motor de Power Query. En Power Query Online también hay indicadores de plegado por paso y un plan de consulta (Microsoft, s. f.-e).

Bloque 4 · Internet de las Cosas (IoT)

I1. Arquitectura IoT por capas y recorrido de una lectura

Una arquitectura IoT se organiza en capas (ITU-T, 2012). El **dispositivo** mide; el **gateway** o edge lo lee, le pone marca de tiempo y lo envía; la **red o broker** transporta los mensajes, casi siempre con MQTT; el **almacenamiento** los guarda, y la **aplicación** los convierte en información. En SolarBI, una lectura nace en el inversor, el gateway la publica cada 5 minutos, llega a Bronze sin cambios, el ETL la valida y la deja en Silver, Gold la suma como energía del día y termina en un panel de Grafana o en una tarjeta de Power BI. Hoy el gateway lo simula el script; la tabla muestra el diseño completo.

Capa	En SolarBI	Dónde queda la lectura
Dispositivo	Inversor de 5 kWp con sensores de irradiancia y temperatura de módulo.	Todavía no está en la base.
Gateway (edge)	Lee el inversor, agrega la marca de tiempo y publica cada 5 minutos.	Mensaje en tránsito.
Red / broker	Broker MQTT con un topic por sitio, dispositivo y variable.	Mensaje en tránsito.
Almacenamiento	PostgreSQL con TimescaleDB, cargado por run_etl.py.	Bronze (crudo) → Silver (validada, 5 min) → Gold (energía diaria).
Aplicación	Grafana para operar y Power BI para analizar.	Grafana lee Silver y Gold; Power BI lee Gold.

I2. MQTT: publish/subscribe, broker, topics y QoS

MQTT es un protocolo de mensajería liviano basado en **publish/subscribe**: los dispositivos publican en un topic y los consumidores se suscriben a él sin conocerse entre sí. En el medio está el **broker**, que recibe cada mensaje y lo reparte a los suscriptores. El **topic** es una ruta jerárquica separada por /. **QoS 0** entrega como máximo una vez, sin confirmación; **QoS 1**, al menos una vez, con posibles duplicados; **QoS 2**, exactamente una vez, a costa de más mensajes de control (OASIS, 2019). Usaría QoS 1, porque el ETL ya elimina duplicados. Para la planta propongo una jerarquía por sitio, dispositivo y variable (pascualbravo/solar/PB-01/inv-1/p_ac_kw); para leer la potencia de todos los inversores basta suscribirse con comodines (pascualbravo/solar/+/+/p_ac_kw).

I3. Pila abierta de monitoreo IoT: Mosquitto, Telegraf, base de series de tiempo y Grafana

Mosquitto es el broker MQTT: recibe lo que publican los gateways (Eclipse Foundation, s. f.). **Telegraf** es un agente de recolección: se suscribe a los topics, convierte cada mensaje en una métrica y la escribe en la base (InfluxData, s. f.). La **base de series de tiempo** guarda las lecturas ordenadas por tiempo y responde rápido a consultas por rango; InfluxDB es una opción, y TimescaleDB es una extensión que convierte una tabla de PostgreSQL en hypertable. **Grafana** consulta esa base y muestra paneles y alertas. PostgreSQL encaja justo ahí: con TimescaleDB, una sola base guarda la serie de 5 minutos y el modelo estrella, y Grafana y Power BI leen el mismo dato gobernado.

I4. Procesamiento por lotes frente a streaming

En **batch** los datos se acumulan y se procesan juntos en un horario; en **streaming** cada evento se procesa apenas llega. Al agregar las lecturas de 5 minutos en fact_energia_dia gano consultas rápidas, una tabla pequeña y cifras fáciles de comparar entre días o inversores. Pierdo el detalle: en el total diario no se ve que el inversor se apagó 20 minutos a mediodía, ni a qué hora pasó. Por eso guardo las dos granularidades (Kimball y Ross, 2013): Silver de 5 minutos para Grafana, que refresca cada pocos segundos y dispara alertas, y Gold diario para Power BI, que sirve para tendencias y metas. El ETL de SolarBI es batch; el streaming llegaría con MQTT.

Bloque 5 · Control de versiones

V1. Git, GitHub, .pbix frente a .pbip y versionado de Grafana

Git es el sistema de control de versiones que guarda la historia del proyecto; **GitHub** es el servicio que aloja repositorios Git y suma colaboración y CI (Chacon y Straub, 2014). Un **repository** es el proyecto con su historia; un **commit**, una foto de los cambios con su mensaje; una **branch**, una línea de trabajo paralela, y un **pull request**, la propuesta de unir una rama con revisión. El .pbix es binario: Git no puede mostrar qué cambió ni fusionarlo. El .pbip guarda el modelo y el informe como texto, así cada medida se ve en el diff (Microsoft, s. f.-f). En Grafana, el dashboard se exporta como JSON y se carga con provisioning (Grafana Labs, s. f.-h).

Parte B · Práctica guiada: los mismos datos en dos herramientas

El código completo está en el repositorio. Aquí va una captura por paso con una explicación corta.

Paso 1 · Bronze: datos crudos

```
OT-bronze-csv X + v
ts,dispositivo_id,p_ac_kw,irradiacion_wm2,temp_modulo_c
2026-10-05 00:00:00,1,0.0,0.0,22.0
2026-10-05 00:05:00,1,0.0,0.0,22.0
2026-10-05 00:10:00,1,0.0,0.0,22.0
2026-10-05 00:15:00,1,0.0,0.0,22.0
2026-10-05 00:20:00,1,0.0,0.0,22.0
2026-10-05 00:25:00,1,0.0,0.0,22.0
2026-10-05 00:30:00,1,0.0,0.0,22.0
2026-10-05 00:35:00,1,0.0,0.0,22.0
2026-10-05 00:40:00,1,0.0,0.0,22.0
2026-10-05 00:45:00,1,0.0,0.0,22.0
2026-10-05 00:50:00,1,0.0,0.0,22.0
Total de líneas: 880
--- Ejemplos de anomalías (potencia negativa) ---
2026-10-05 04:40:00,1,-1.0,0.0,22.0
2026-10-05 08:10:00,1,-1,534.6,38.1
2026-10-05 15:05:00,1,-1,634.5,42.7
PS D:\SEMESTRE 2026-2\Inteligencia de Negocios BI\solarbi-romero-dorado>
```

Figura 1. *Primeras filas de data/bronze/telemetry.csv, total de líneas y ejemplos de potencia negativa.*

El simulador del curso genera data/bronze/telemetry.csv con tres días de lecturas cada 5 minutos de un inversor de 5 kWp. El archivo tiene 880 líneas (879 lecturas más el encabezado) y no se modifica: ya trae anomalías como potencias de -1 kW y la irradiancia vacía de las 00:15.

Paso 2 · Silver: reglas de calidad

[illegible]

Figura 2. Reporte de calidad de la primera carga (evidencia versionada en docs/evidencias/fase-2-ejecucion.md).

etl/run_etl.py aplica las reglas del contrato de datos (contracts/telemetry.yaml): rango físico de la potencia, duplicados por fecha y dispositivo, y datos faltantes, entre otras. De 879 filas leídas rechazó 22 por rango, quitó 15 duplicadas y marcó 23 sin irradiancia; quedaron 842 válidas (95,79 %). La captura sale de la evidencia guardada en el repositorio, porque esa primera carga no se puede repetir sobre la misma base.

Paso 3 · Gold: carga en PostgreSQL

```
03a-eti x + v
[1/8] Contrato y archivo ok 0.03 s
contrato telemetria v1.1.0; archivo data/bronze/telemetry.csv
sha256 bca6d50ed9a61c9fae7cd697c453c0faa8d70024eb25dd5d70a1a258b6f4a69
[2/8] Abrir corrida en dq.etl_run_log ok 0.01 s
run_id 91d513db-7a3d-416b-b646-28ab7e52eb2e
[3/8] Bronze ok 0.01 s
archivo ya cargado (mismo sha256): se reutilizan sus filas de Bronze de la corrida 3ad88b48-5448-4828-a32d-69f9f37ff999
[4/8] Silver (reglas de calidad + UPSERT) ok 0.15 s
[5/8] Dimensiones ok 0.02 s
[6/8] Gold (dwh.fact_energia_dia por día local) ok 0.02 s
[7/8] Fallas (dq.fault_event) ok 0.03 s
[8/8] Cerrar corrida (sucedido) ok 0.00 s

Reporte de calidad (Paso 2)
Corrida : 91d513db-7a3d-416b-b646-28ab7e52eb2e
Archivo : data/bronze/telemetry.csv (contrato v1.1.0)
Bronze : omitido: archivo ya cargado (filas de la corrida 3ad88b48-5448-4828-a32d-69f9f37ff999)
Filas leídas : 879
Rechazadas por regla : missing_key=0, invalid_format=0, unknown_device=0, off_grid=0, range_p_ac_kw=22, missing_p_ac_kw=0
Rechazadas (distintas) : 22
Deduplicadas : 15
Marcadas (flag) : snapped_to_grid=0, missing_irradiancia=23, range_irradiancia=0, range_temp_modulo=0 (23 filas)
Filas válidas : 842
% datos válidos : 95.79 %
Días cargados en Gold : 3 (energía total = 80.956 kwh)
Eventos de falla : 0

Nota: los conteos por regla pueden solaparse (una fila puede fallar varias reglas).
Cuadre: leídas = válidas + rechazadas distintas + deduplicadas -> 879 = 842 + 22 + 15 (OK)
PS D:\SEMESTRE 2026-2\Inteligencia de Negocios BI\solarbi-romero-dorado>
```

Figura 3. Segunda ejecución de run_etl con el mismo archivo: Bronze omitido.

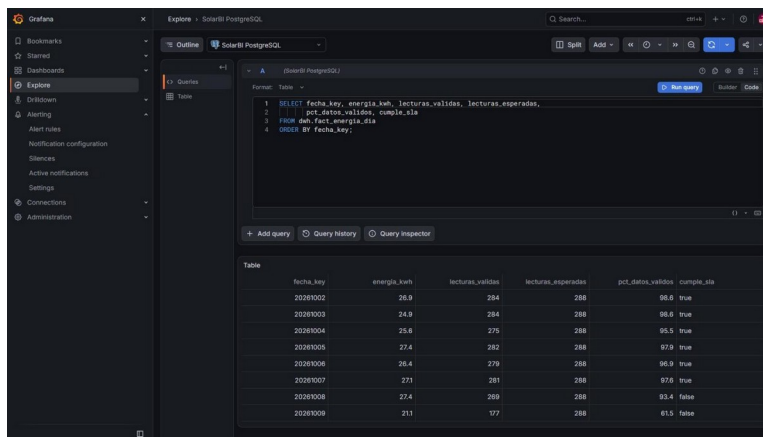
Al correr otra vez el mismo comando, el ETL reconoce el archivo por su hash SHA-256, omite Bronze y recalcula Silver y Gold con UPSERT. El reporte da exactamente las mismas cifras de la primera carga.

```
03b-conteos x + v
tabla      filas
bronze.telemetry_raw  3350
silver.lectura_5min    2131
dwh.fact_energia_dia   8
dq.fault_event         4
dq.etl_run_log        34

fecha_key disp.  energia_kwh  lecturas  % válidos
20261002  1      26.9448    284      98.61
20261003  1      24.9322    284      98.61
20261004  1      25.5603    275      95.49
20261005  1      27.4233    282      97.92
20261006  1      26.3838    279      96.88
20261007  1      27.1406    281      97.57
20261008  1      27.4413    269      93.40
20261009  1      21.0657    177      61.46
PS D:\SEMESTRE 2026-2\Inteligencia de Negocios BI\solarbi-romero-dorado>
```

Figura 4. Conteos por tabla y energía por día después de la segunda ejecución.

Los conteos no cambian: Bronze 3350, Silver 2131 y 8 días en Gold, con la misma energía por día. Solo crece dq.etl_run_log, que guarda una fila por corrida. Así se comprueba que la carga es idempotente. Las cifras superan las del archivo del simulador (879 filas, del 5 al 7 de octubre) porque la base también guarda la muestra de fallas del 2 al 4 y las réplicas en vivo del 8 y el 9.



The screenshot shows the Grafana Explore interface with a SQL query and its results. The query is:

```
SELECT fecha_key, energia_kwh, lecturas_validos, lecturas Esperadas,
       pct_datos_validos, cumple_sla
FROM dwh.fact_energia_dia
ORDER BY fecha_key;
```

The results table shows data for dates from 20261002 to 20261009. The columns are: fecha_key, energia_kwh, lecturas_validos, lecturas Esperadas, pct_datos_validos, and cumple_sla.

fecha_key	energia_kwh	lecturas_validos	lecturas Esperadas	pct_datos_validos	cumple_sla
20261002	26.9	284	288	98.6	true
20261003	24.9	284	288	98.6	true
20261004	25.6	275	288	95.5	true
20261005	27.4	282	288	97.9	true
20261006	26.4	279	288	96.9	true
20261007	27.1	281	288	97.6	true
20261008	27.4	269	288	93.4	false
20261009	21.1	177	288	61.5	false

Figura 5. dwh.fact_energia_dia consultada desde Grafana Explore con el usuario de solo lectura.

La energía de cada intervalo es la potencia por 5/60 de hora y Gold la suma por día. El 6 de octubre da 26,3838 kWh con 96,88 % de datos válidos; en la tabla, Grafana redondea a un decimal.

La programación diaria quedó escrita, no instalada. En Linux sería una línea de cron que lanza el ETL todos los días a la medianoche (0 0 * * * cd /ruta/solarbi-romero-dorado && .venv/bin/python -m etl.run_etl --file data/bronze/telemetry.csv); en Windows, una tarea diaria del Programador de tareas a las 00:00 que ejecuta el mismo comando (schtasks /Create /SC DAILY /ST 00:00 /TN SolarBI_ETL /TR "<ruta>\.venv\Scripts\python.exe -m etl.run_etl --file <ruta>\data\bronze\telemetry.csv").

Paso 4 · Power BI

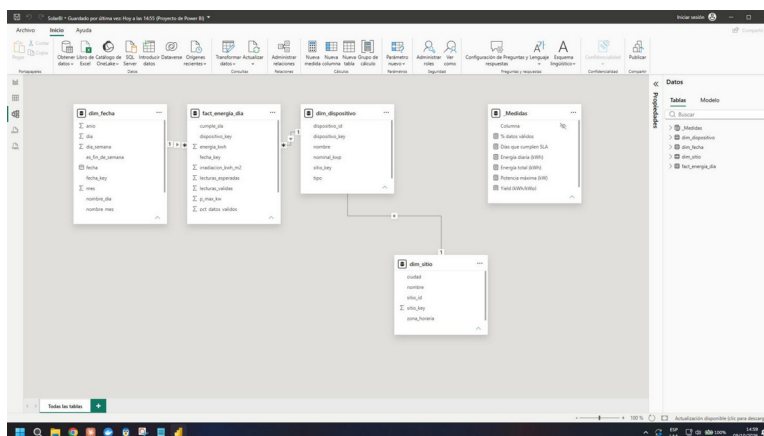


Figura 6. Vista Modelo: esquema estrella y tabla _Medidas.

Power BI importa las cuatro tablas de dwh con el usuario powerbi_reader. El modelo es una estrella: fact_energia_dia se relaciona de muchos a uno con dim_fecha y dim_dispositivo, y esta con dim_sitio. Las medidas viven en la tabla _Medidas: la energía total suma la energía de la tabla de hechos (SUM(fact_energia_dia[energia_kwh])), la energía diaria reutiliza esa medida en el gráfico y el porcentaje de datos válidos divide las lecturas válidas entre las esperadas (DIVIDE(SUM(lecturas_validas), SUM(lecturas_esperadas))).

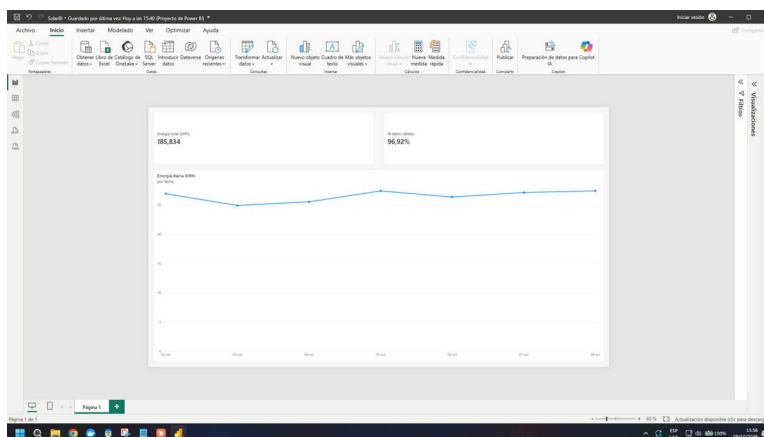


Figura 7. Página del informe con las dos tarjetas y la energía diaria del 2 al 8 de octubre.

Las tres cifras salen de medidas DAX, no de columnas arrastradas. El porcentaje es un cociente de sumas para que un día incompleto no pese igual que uno completo. Filtré la página hasta el 8 de octubre porque el 9 fue el día de la demostración en vivo y quedó incompleto: 185,834 kWh y 96,92 %.

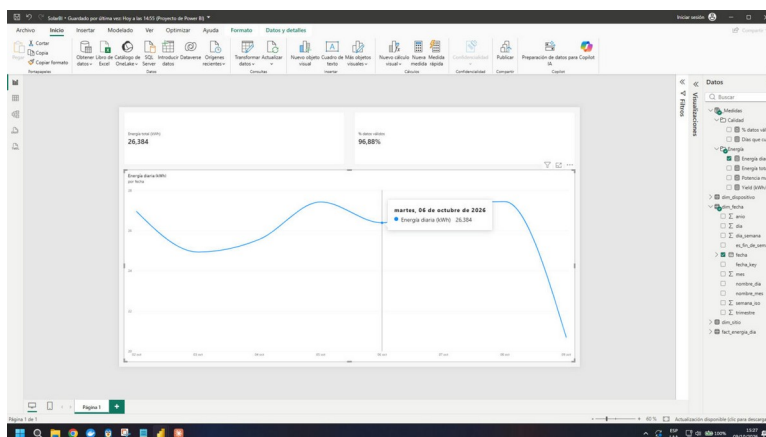


Figura 8. Comprobación cruzada: tarjeta filtrada al 6 de octubre.

Al seleccionar el 6 de octubre en el gráfico, la tarjeta muestra 26,384 kWh, la misma cifra que el panel de Grafana para ese día. Las dos herramientas leen la misma base y llegan al mismo número.

Paso 5 · Grafana



Figura 9. Tablero SolarBI · Operación de la planta, últimos 7 días, día seleccionado 2026-10-06.

Grafana OSS corre en Docker y el tablero se carga como código desde grafana/dashboards/solarbi-operacion.json, que es también el JSON exportado. El panel de potencia usa \$__timeFilter(ts) sobre silver.lectura_5min; el stat muestra la energía del día elegido en la variable Día (Gold): 26,384 kWh el 6 de octubre.

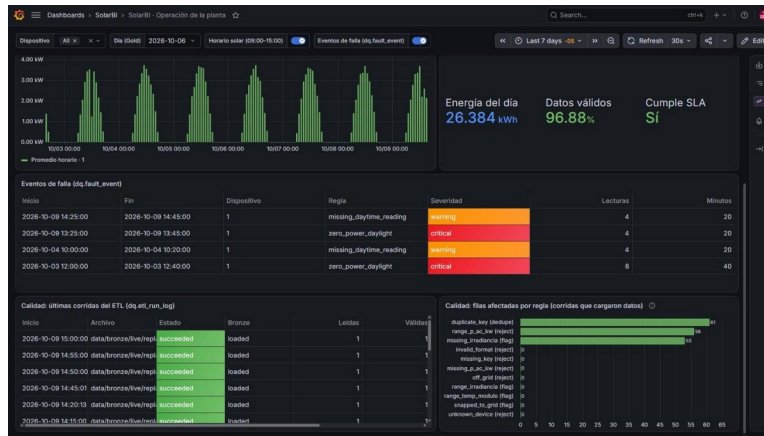


Figura 10. Parte inferior del tablero: eventos de falla y paneles de calidad de datos.

Abajo están los eventos de falla que detecta el ETL y dos paneles de calidad: las últimas corridas (dq.etl_run_log) y las filas afectadas por cada regla.

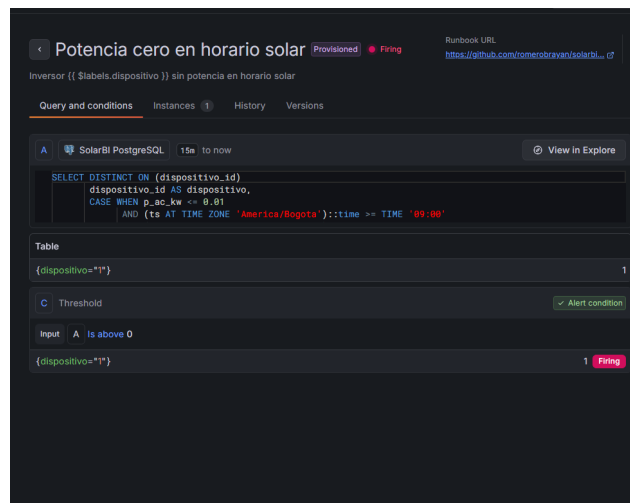


Figura 11. Regla «Potencia cero en horario solar» en estado Firing.

La regla se evalúa cada minuto: si la última lectura de un inversor es de 0,01 kW o menos entre las 9:00 y las 15:00 y sigue así 10 minutos, pasa a Firing. La probé con una réplica en vivo en la que detuve el inversor.

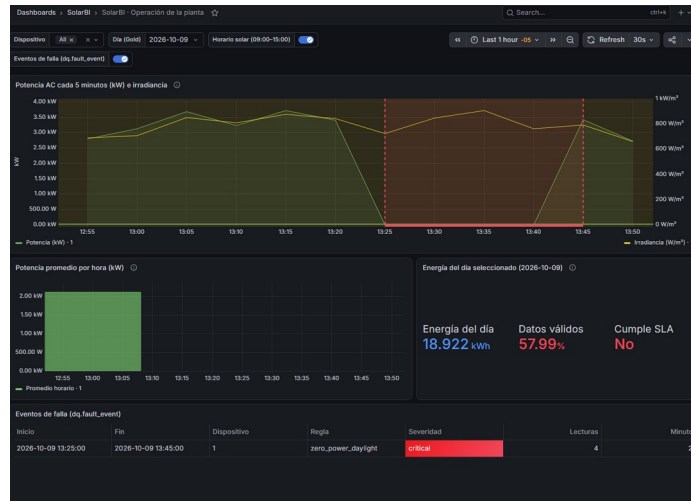


Figura 12. Caída de potencia durante la réplica: la potencia llega a cero mientras hay irradiancia.

En el tablero se ve el disparo: la potencia cae a cero a las 13:25 con la irradiancia todavía alta, y el evento queda registrado como `zero_power_daylight` con severidad crítica.

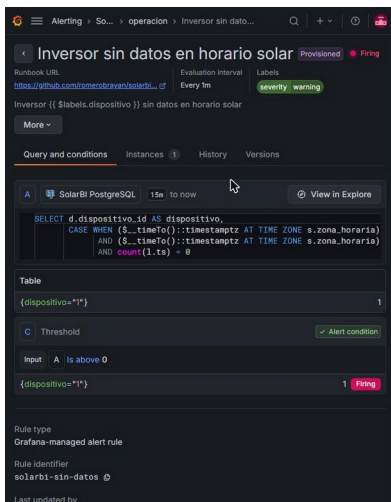


Figura 13. Segunda regla: «Inversor sin datos en horario solar».

Agregué una segunda regla porque la primera no ve un inversor que deja de enviar lecturas: sin filas no hay nada que comparar. Esta parte del catálogo de dispositivos y avisa si pasan 15 minutos sin datos en horario solar.

Paso 6 · Reflexión

Con los mismos datos, Grafana resolvió mejor lo operativo: ver la potencia cada 5 minutos, detectar el disparo del inversor y avisar sin que nadie esté mirando la pantalla. Power BI resolvió mejor el análisis: con el modelo estrella y las medidas DAX, el mismo número (26,384 kWh el 6 de octubre) se puede cortar por día, inversor o sitio sin escribir otra consulta. Lo que más me costó en Grafana fue ajustar las alertas para que no se dispararan de noche ni por una sola lectura mala; en Power BI, entender por qué la relación con la fecha salía 1 a 1 y dejar el modelo como estrella. Para el operador de la

planta usaría Grafana, porque necesita reaccionar en minutos; para un directivo, Power BI, porque le interesa la energía y el ahorro del mes frente a la meta.

Parte C · Repositorio en GitHub

Enlace: <https://github.com/romerobrayan/solarbi-romero-dorado>

El repositorio es público y sigue la estructura pedida: README.md, .gitignore, docs/, data/ (bronze y silver), etl/, sql/, powerbi/ (proyecto .pbip) y grafana/ (dashboard JSON). Las credenciales viven en un .env excluido del repositorio; solo se versiona .env.example. Cada push corre las pruebas en GitHub Actions.

Desarrollé el trabajo de forma individual, por eso el repositorio no tiene un segundo colaborador ni commits de otro autor.

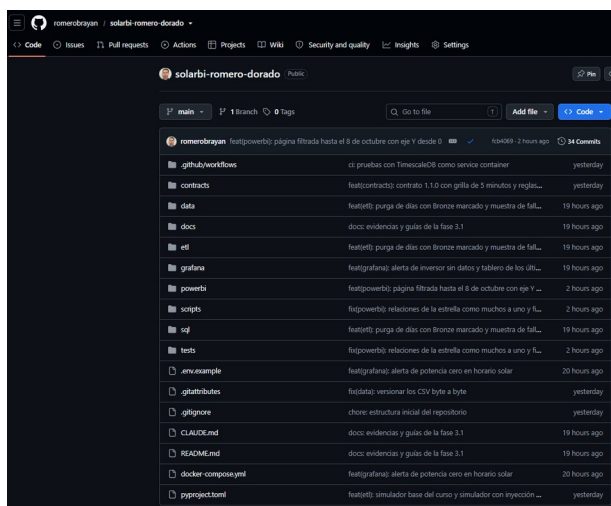


Figura 14. Página principal del repositorio público con su estructura de carpetas.

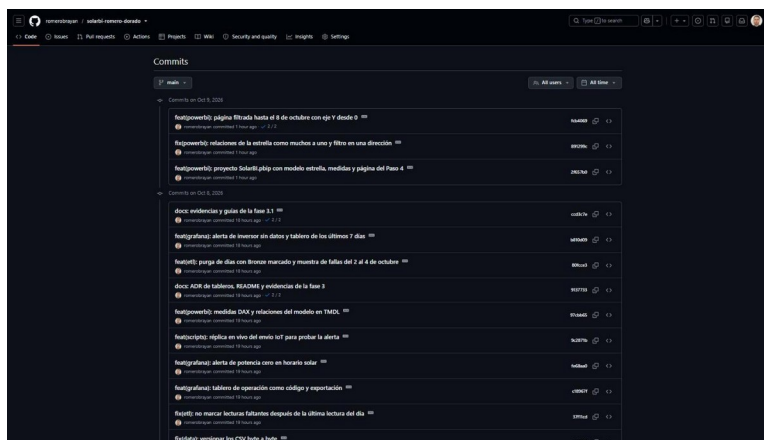


Figura 15. Historial de commits de main con mensajes descriptivos.

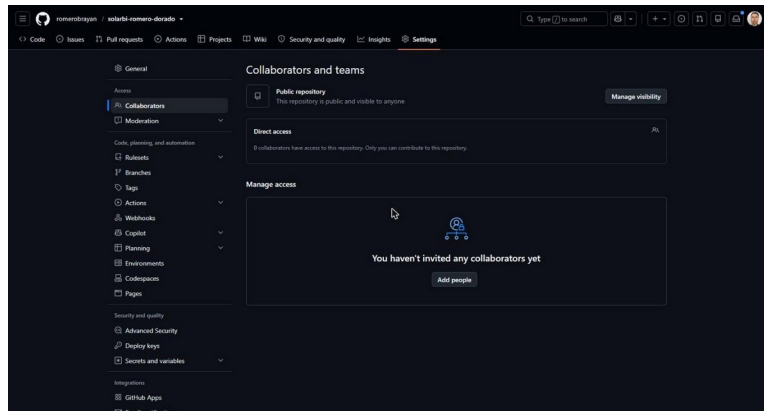


Figura 16. Settings → Collaborators: repositorio público, sin colaboradores adicionales.

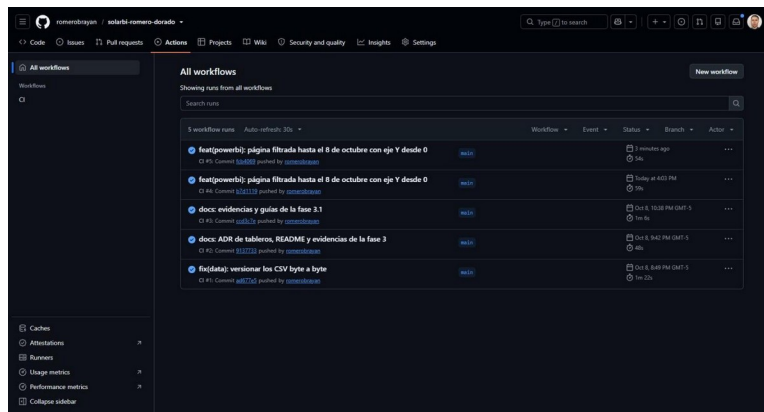


Figura 17. GitHub Actions: todas las ejecuciones de CI terminaron con éxito.

Referencias

- Apache Software Foundation. (s. f.). *What is Airflow?* Apache Airflow documentation. Recuperado el 9 de octubre de 2026, de <https://airflow.apache.org/docs/apache-airflow/stable/index.html>
- Chacon, S. y Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. <https://git-scm.com/book/es/v2>
- Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.
- DAMA International. (2017). *DAMA-DMBOK: Data management body of knowledge* (2.^a ed.). Technics Publications.
- Databricks. (s. f.). *What is a medallion architecture?* Recuperado el 9 de octubre de 2026, de <https://www.databricks.com/glossary/medallion-architecture>
- Eclipse Foundation. (s. f.). *Eclipse Mosquitto: An open source MQTT broker*. Recuperado el 9 de octubre de 2026, de <https://mosquitto.org/>
- Few, S. (2006). *Information dashboard design: The effective visual communication of data*. O'Reilly Media.

- Grafana Labs. (s. f.-a). *Alerting fundamentals*. Grafana documentation. Recuperado el 8 de octubre de 2026, de <https://grafana.com/docs/grafana/latest/alerting/fundamentals/>
- Grafana Labs. (s. f.-b). *Dashboards overview*. Grafana documentation. Recuperado el 8 de octubre de 2026, de <https://grafana.com/docs/grafana/latest/fundamentals/dashboards-overview/>
- Grafana Labs. (s. f.-c). *Licensing*. Recuperado el 8 de octubre de 2026, de <https://grafana.com/licensing/>
- Grafana Labs. (s. f.-d). *Pricing*. Recuperado el 8 de octubre de 2026, de <https://grafana.com/pricing/>
- Grafana Labs. (s. f.-e). *Roles and permissions*. Grafana documentation. Recuperado el 8 de octubre de 2026, de <https://grafana.com/docs/grafana/latest/administration/roles-and-permissions/>
- Grafana Labs. (s. f.-f). *Variables*. Grafana documentation. Recuperado el 8 de octubre de 2026, de <https://grafana.com/docs/grafana/latest/dashboards/variables/>
- Grafana Labs. (s. f.-g). *PostgreSQL query editor*. Grafana documentation. Recuperado el 9 de octubre de 2026, de <https://grafana.com/docs/grafana/latest/datasources/postgres/query-editor/>
- Grafana Labs. (s. f.-h). *Provision Grafana*. Grafana documentation. Recuperado el 9 de octubre de 2026, de <https://grafana.com/docs/grafana/latest/administration/provisioning/>
- InfluxData. (s. f.). *Telegraf documentation*. Recuperado el 9 de octubre de 2026, de <https://docs.influxdata.com/telegraf/>
- ITU-T. (2012). *Overview of the Internet of things* (Recomendación UIT-T Y.2060). Unión Internacional de Telecomunicaciones.
- Kimball, R. y Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3.^a ed.). Wiley.
- Marion, B., Adelstein, J., Boyle, K., Hayden, H., Hammond, B., Fletcher, T., Canada, B., Narang, D., Kimber, A., Mitchell, L., Rich, G. y Townsend, T. (2005). *Performance parameters for grid-connected PV systems* (NREL/CP-520-37358). National Renewable Energy Laboratory.
- Microsoft. (2024). *Announcing the retirement of real-time streaming in Power BI*. Microsoft Fabric Community. https://community.fabric.microsoft.com/blog/fbc_pbiupdatesblog/announcing-the-retirement-of-real-time-streaming-in-power-bi/5174368
- Microsoft. (s. f.-a). *Data refresh in Power BI*. Microsoft Learn. Recuperado el 8 de octubre de 2026, de <https://learn.microsoft.com/en-us/power-bi/connect-data/refresh-data>
- Microsoft. (s. f.-b). *Power BI pricing*. Recuperado el 8 de octubre de 2026, de <https://www.microsoft.com/en-us/power-platform/products/power-bi/pricing>

- Microsoft. (s. f.-c). *Set data alerts in the Power BI service*. Microsoft Learn. Recuperado el 8 de octubre de 2026, de <https://learn.microsoft.com/en-us/power-bi/create-reports/service-set-data-alerts>
- Microsoft. (s. f.-d). *Row-level security (RLS) with Power BI*. Microsoft Learn. Recuperado el 9 de octubre de 2026, de <https://learn.microsoft.com/en-us/fabric/security/service-admin-row-level-security>
- Microsoft. (s. f.-e). *Overview of query evaluation and query folding in Power Query*. Microsoft Learn. Recuperado el 9 de octubre de 2026, de <https://learn.microsoft.com/en-us/power-query/query-folding-basics>
- Microsoft. (s. f.-f). *Power BI Desktop projects (PBIP)*. Microsoft Learn. Recuperado el 9 de octubre de 2026, de <https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-overview>
- OASIS. (2019). *MQTT Version 5.0* (OASIS Standard). <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- PostgreSQL Global Development Group. (s. f.). *INSERT*. PostgreSQL documentation. Recuperado el 9 de octubre de 2026, de <https://www.postgresql.org/docs/current/sql-insert.html>